
UNIVERSIDAD NACIONAL AUTÓNOMA DE HONDURAS

Campus Comayagua (UNAH-CURC)

Asignatura: Inteligencia Artificial



AgroDetect

Sistema Inteligente para la De- tección de Enfermedades y Pla- gas Foliares en el Cultivo de Café

*Un sistema de Inteligencia Artificial apli-
cado a la caficultura de Comayagua, Honduras*

Informe Final de Proyecto

Integrantes

Jeimy Jazmín Palma Santos 20221900269

Ángeles Izamar Euceda Herrera 20221930061

Kilver Said Nolasco Parada 20221930129

Docente: Asalia Alejandra Zavala

Fecha: agosto de 2026

Repositorio: github.com/Izamar-0302/AgroDetect

Índice

1	Introducción	3
2	Planteamiento del problema	3
3	Justificación	4
4	Objetivos	5
5	Marco teórico	5
5.1	Inteligencia Artificial y Visión Artificial	6
5.2	Redes Neuronales Convolucionales (CNN)	6
5.3	Transfer Learning	6
5.4	Data Augmentation	6
5.5	Interpretabilidad de modelos: Grad-CAM	7
5.6	Métricas de evaluación	7
6	Metodología propuesta	7
6.1	Descripción del proceso	7
6.2	Datos utilizados	8
6.3	Algoritmos y técnicas de Inteligencia Artificial implementadas	11
6.4	Herramientas y tecnologías empleadas	12
7	Diseño de la solución	12
7.1	Arquitectura del sistema	12
7.2	Diagrama de funcionamiento	13
7.3	Base de datos	13
7.4	Flujo del sistema	13
8	Desarrollo e implementación	14
8.1	Módulos principales	14
8.2	Fragmentos de código relevantes	14
8.3	Justificación de las decisiones técnicas	15
9	Resultados obtenidos	16
9.1	Comparación de arquitecturas	16
9.2	Curvas de entrenamiento	17
9.3	Desempeño del modelo final	18

9.4	Interpretabilidad del modelo y ronda de ajuste fino (Grad-CAM)	20
9.5	Umbral de decisión	22
9.6	Pruebas realizadas	24
10	Beneficios del proyecto	24
11	Usuarios beneficiados	25
12	Escalabilidad de la solución	25
13	Mantenimiento de la aplicación	25
13.1	Operación continua	25
13.2	Estrategia de actualización del modelo	25
13.3	Mantenimiento preventivo y correctivo	26
13.4	Respaldo de la información	26
14	Conclusiones	26
15	Recomendaciones	27
16	Referencias bibliográficas	28
17	Anexos	29

1 Introducción

El café es uno de los pilares de la economía hondureña: Honduras se ubica entre los principales exportadores de Centroamérica y Latinoamérica, este rubro genera ingresos directos e indirectos para cientos de miles de familias, incluidas numerosas comunidades productoras del departamento de Comayagua. Sin embargo, la caficultura hondureña enfrenta una amenaza recurrente y creciente: las enfermedades foliares, en particular la **roya del café** (*Hemileia vastatrix*), que provoca la caída prematura de las hojas, debilita la planta y, en casos severos de defoliación extrema, puede causar la muerte del cafeto. A esta se suman otras afecciones comunes como la *cercospora* y la *phoma*, así como plagas que también dañan visiblemente el follaje, como la *araña roja* (*red spider mite*) y el *minador de la hoja* (*leaf miner*); todas ellas reducen el rendimiento y la calidad del grano.

De acuerdo con reportes del Instituto Hondureño del Café (IHCAFE), la incidencia promedio nacional de la roya continúa siendo motivo de alerta y una proporción considerable de las fincas muestreadas presenta niveles de infección altos. Históricamente, la crisis de roya de los ciclos 2012-2013 provocó una caída superior al 20 % en el volumen exportado de café hondureño, con fuertes pérdidas económicas y de empleo en las zonas rurales cafetaleras.

Este informe documenta el desarrollo completo de **AgroDetect**, un sistema de Inteligencia Artificial que evolucionó desde un anteproyecto inicial hasta una solución funcional: un modelo de visión artificial entrenado y evaluado de forma iterativa, integrado en una aplicación web construida con Streamlit, capaz de diagnosticar el estado de una hoja de café a partir de una fotografía y ofrecer recomendaciones de manejo agronómico.

2 Planteamiento del problema

En Comayagua, al igual que en gran parte de las zonas cafetaleras de Honduras, la mayoría de los productores son pequeños y medianos caficultores que gestionan sus fincas de manera artesanal, con acceso limitado y esporádico a técnicos especializados del IHCAFE u otras instituciones de extensión agrícola. El diagnóstico de enfermedades y plagas foliares se realiza casi exclusivamente por observación directa, un proceso que:

- ❑ Depende de la experiencia y capacitación previa del productor, lo cual genera diagnósticos inconsistentes.
- ❑ Retrasa la toma de decisiones sobre el manejo fitosanitario (aplicación de fungicidas, poda, renovación de plantas), lo que agrava el impacto de la enfermedad.
- ❑ Se dificulta aún más en fincas de difícil acceso geográfico, comunes en la topografía mon-

tañosa de Comayagua.

Problema central: Los pequeños y medianos productores de café de Comayagua carecen de una herramienta accesible, rápida y confiable que les permita identificar las principales enfermedades y plagas foliares del cafeto (roya, cercospora, phoma, araña roja y minador de la hoja), lo que provoca diagnósticos tardíos, aumento en la severidad de las infecciones e infestaciones, y pérdidas económicas evitables.

3 Justificación

La Inteligencia Artificial, particularmente las técnicas de **visión artificial** y **aprendizaje profundo** (*Deep Learning*), ha demostrado ser altamente efectiva para la identificación automática de enfermedades y plagas en hojas de cultivos a partir de imágenes, mediante redes neuronales convolucionales (CNN) y arquitecturas de transferencia de aprendizaje como MobileNet, EfficientNet o ResNet.

El uso de IA para este problema se justifica por las siguientes razones:

- ❑ Permite un **diagnóstico objetivo y consistente**, reduciendo la dependencia exclusiva del juicio visual del productor.
- ❑ Es **escalable**: una vez entrenado, el modelo puede analizar miles de imágenes sin fatiga ni variabilidad de criterio.
- ❑ Puede desplegarse en una **aplicación accesible desde un teléfono inteligente**, herramienta cada vez más presente entre los productores de café de Honduras, incluso en zonas rurales.
- ❑ Contribuye a la **reducción de pérdidas económicas** al permitir intervenciones fitosanitarias más oportunas y focalizadas.
- ❑ Se alinea con los esfuerzos institucionales de IHCAFE en materia de monitoreo permanente de plagas y enfermedades, aportando una herramienta complementaria de bajo costo.

4 Objetivos

Objetivo general

Desarrollar y desplegar un sistema basado en Inteligencia Artificial, utilizando técnicas de visión artificial y aprendizaje profundo, para la detección y clasificación de enfermedades y plagas foliares en el cultivo de café, accesible mediante una aplicación web, como herramienta de apoyo al diagnóstico fitosanitario de los productores de la región de Comayagua, Honduras.

Objetivos específicos

- ❑ Recopilar y curar un conjunto de datos a partir de repositorios públicos de imágenes de hojas de café sanas y afectadas por las cinco clases priorizadas: roya, cercospora, phoma, araña roja y minador de la hoja.
- ❑ Diseñar, entrenar y **comparar formalmente distintas arquitecturas** de redes neuronales convolucionales (CNN) mediante *transfer learning*, seleccionando la más adecuada según su desempeño y su viabilidad de despliegue.
- ❑ Evaluar el desempeño del modelo utilizando métricas estándar (exactitud, precisión, sensibilidad y F1-score) y técnicas de interpretabilidad (Grad-CAM) sobre un conjunto de prueba independiente.
- ❑ Diagnosticar y corregir con evidencia visual, las causas de confusión entre clases visualmente similares.
- ❑ Desarrollar una interfaz web con Streamlit que permita a los productores subir o capturar una fotografía de una hoja de café y recibir un diagnóstico automatizado.
- ❑ Integrar un módulo de recomendaciones basado en un modelo de lenguaje (LLM), consumido mediante una API key desde la propia aplicación, que a partir del diagnóstico entregue al productor sugerencias prácticas de manejo agronómico.
- ❑ Desplegar la solución completa en una plataforma en la nube de acceso gratuito (Streamlit Community Cloud).
- ❑ Definir una estrategia de mantenimiento, actualización y mejora continua del sistema a partir de la retroalimentación de los usuarios finales.

5 Marco teórico

5.1 Inteligencia Artificial y Visión Artificial

La **Inteligencia Artificial (IA)** es el campo de la informática que desarrolla sistemas capaces de realizar tareas que normalmente requieren inteligencia humana, como reconocer patrones visuales. La **visión artificial** (*Computer Vision*) es la subárea de la IA enfocada en que una computadora interprete y comprenda el contenido de imágenes o video, y es la base técnica sobre la que se construye AgroDetect.

5.2 Redes Neuronales Convolucionales (CNN)

Las **redes neuronales convolucionales** son la arquitectura de referencia para tareas de clasificación de imágenes. A diferencia de una red neuronal totalmente conectada, una CNN aplica filtros (convoluciones) que aprenden a detectar patrones visuales locales – bordes, texturas, manchas de color – y los combinan en capas sucesivas para reconocer estructuras cada vez más complejas, hasta llegar a una capa final que clasifica la imagen en una de varias categorías.

5.3 Transfer Learning

El **aprendizaje por transferencia** (*transfer learning*) consiste en reutilizar una red neuronal ya entrenada sobre un conjunto de datos masivo y genérico (en este proyecto, ImageNet, con más de un millón de imágenes de mil categorías distintas) como punto de partida para una tarea nueva y más específica. En vez de aprender desde cero a reconocer bordes y texturas, el modelo parte de una base que ya "sabe ver", y solo se ajustan las últimas capas para la tarea concreta (clasificar hojas de café). Esto permite obtener buen desempeño incluso con conjuntos de datos de tamaño moderado, y reduce drásticamente el tiempo y el costo computacional de entrenamiento frente a entrenar una red desde cero. En AgroDetect, cada arquitectura candidata se entrenó en dos fases: primero con la base preentrenada completamente congelada (solo se ajusta el clasificador final), y luego con un bloque de capas superiores de la base descongeladas para un ajuste fino (*fine-tuning*) a una tasa de aprendizaje baja.

5.4 Data Augmentation

El **aumento de datos** (*data augmentation*) es una técnica que genera variaciones artificiales de las imágenes de entrenamiento (rotaciones, volteos, ajustes de brillo, contraste y zoom) para exponer al modelo a mayor variabilidad visual sin necesidad de recolectar más fotografías reales, mitigando el sobreajuste (*overfitting*) y mejorando la capacidad de generalización del modelo.

5.5 Interpretabilidad de modelos: Grad-CAM

Grad-CAM (*Gradient-weighted Class Activation Mapping*) es una técnica de interpretabilidad que genera un mapa de calor sobre la imagen de entrada, señalando qué región influyó más en la predicción de un modelo de *Deep Learning*. Se utiliza para verificar que una CNN esté aprendiendo patrones visuales relevantes (por ejemplo: las lesiones de una hoja) y no atajos espurios del conjunto de datos (por ejemplo: el fondo de la fotografía) – un problema conocido como *domain gap* o brecha de dominio, muy común en visión por computador aplicada a agricultura con datasets provenientes de internet – y fue una herramienta central en el diagnóstico y mejora del modelo de este proyecto.

5.6 Métricas de evaluación

Para evaluar un modelo de clasificación se emplean las siguientes métricas, calculadas a partir de la matriz de confusión sobre un conjunto de prueba independiente:

- ❑ **Exactitud** (*accuracy*): proporción de predicciones correctas sobre el total.
- ❑ **Precisión**: de las veces que el modelo predijo una clase, qué proporción era realmente correcta (mide los falsos positivos).
- ❑ **Sensibilidad o *recall***: de todos los casos reales de una clase, qué proporción fue detectada correctamente (mide los falsos negativos).
- ❑ **F1-score**: media armónica entre precisión y sensibilidad, útil como métrica balanceada única, especialmente cuando existe desbalance entre clases.

6 Metodología propuesta

6.1 Descripción del proceso

El desarrollo del proyecto siguió una metodología iterativa inspirada en el ciclo **CRISP-DM** (*Cross Industry Standard Process for Data Mining*), adaptada al desarrollo de soluciones de visión artificial:

1. **Comprensión del problema**: revisión de literatura técnica y material del IHCAFE sobre síntomas y ciclos de las enfermedades y plagas foliares del café.
2. **Recolección y curaduría de datos**: búsqueda, descarga y organización de imágenes desde repositorios públicos, con auditoría visual manual para descartar imágenes de baja calidad.

3. **Preprocesamiento:** limpieza, normalización, redimensionamiento, división estratificada del dataset y aumento de datos.
4. **Modelado:** entrenamiento y **comparación formal** de arquitecturas CNN mediante *transfer learning*.
5. **Evaluación:** matriz de confusión, métricas estándar, curvas ROC y análisis de interpretabilidad (Grad-CAM).
6. **Diagnóstico y mejora:** ronda adicional de ajuste fino guiada por evidencia (refuerzo del peso de la clase con menor desempeño y descongelamiento de capas adicionales de la base convolucional).
7. **Calibración del umbral de decisión:** análisis de la confianza del modelo para definir a partir de qué probabilidad se acepta un diagnóstico como concluyente.
8. **Implementación:** construcción de la aplicación web e integración del modelo y del módulo de recomendaciones.
9. **Despliegue y validación:** publicación en la nube y pruebas piloto, incluyendo pruebas con fotografías reales fuera del conjunto de datos.

6.2 Datos utilizados

Se trabajó con imágenes digitales de hojas de café provenientes de repositorios públicos de acceso abierto (RoCoLe, CLR-Dataset, Roboflow Universe, Kaggle, JMuBEN, entre otros), cubriendo seis clases: hoja sana, roya, cercospora, phoma, araña roja y minador de la hoja. Tras la descarga, extracción y un filtrado automático de imágenes corruptas o ilegibles, el dataset organizado quedó compuesto por **2 745 imágenes**, distribuidas de forma desigual entre clases, como se resume en la Tabla 2 y la Figura 1.

Clase	Tipo	Alcance dentro del proyecto
Roya (<i>Hemileia vastatrix</i>)	Enfermedad	Clasificación de presencia/ausencia.
Cercospora	Enfermedad	Clasificación de presencia/ausencia.
Phoma	Enfermedad	Clasificación de presencia/ausencia.
Araña roja (<i>red spider mite</i>)	Plaga	Clasificación de presencia/ausencia.
Minador de la hoja (<i>leaf miner</i>)	Plaga	Clasificación de presencia/ausencia.

Tabla 1: Enfermedades y plagas priorizadas por el equipo.

Clase	Entrenamiento	Validación	Prueba	Total
Cercospora	411	88	89	588
Phoma	350	75	75	500
Minador	350	75	75	500
Roya	281	60	61	402
Sana	280	60	60	400
Araña roja	248	53	54	355
Total	1 920	411	414	2 745

Tabla 2: Número de imágenes reales por clase y su distribución en la división estratificada 70 %/15 %/15 % (entrenamiento/validación/prueba).

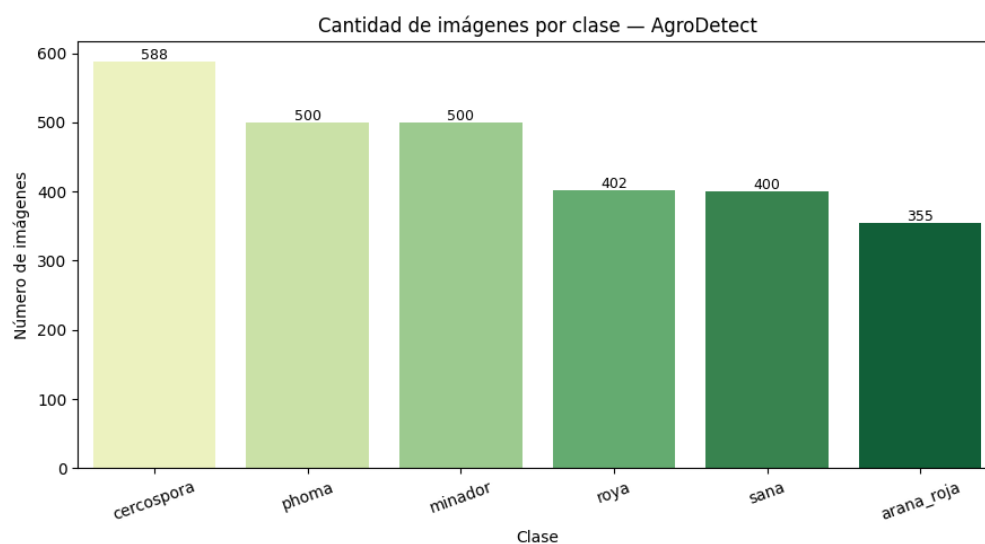


Figura 1: Cantidad de imágenes reales por clase en el dataset organizado.

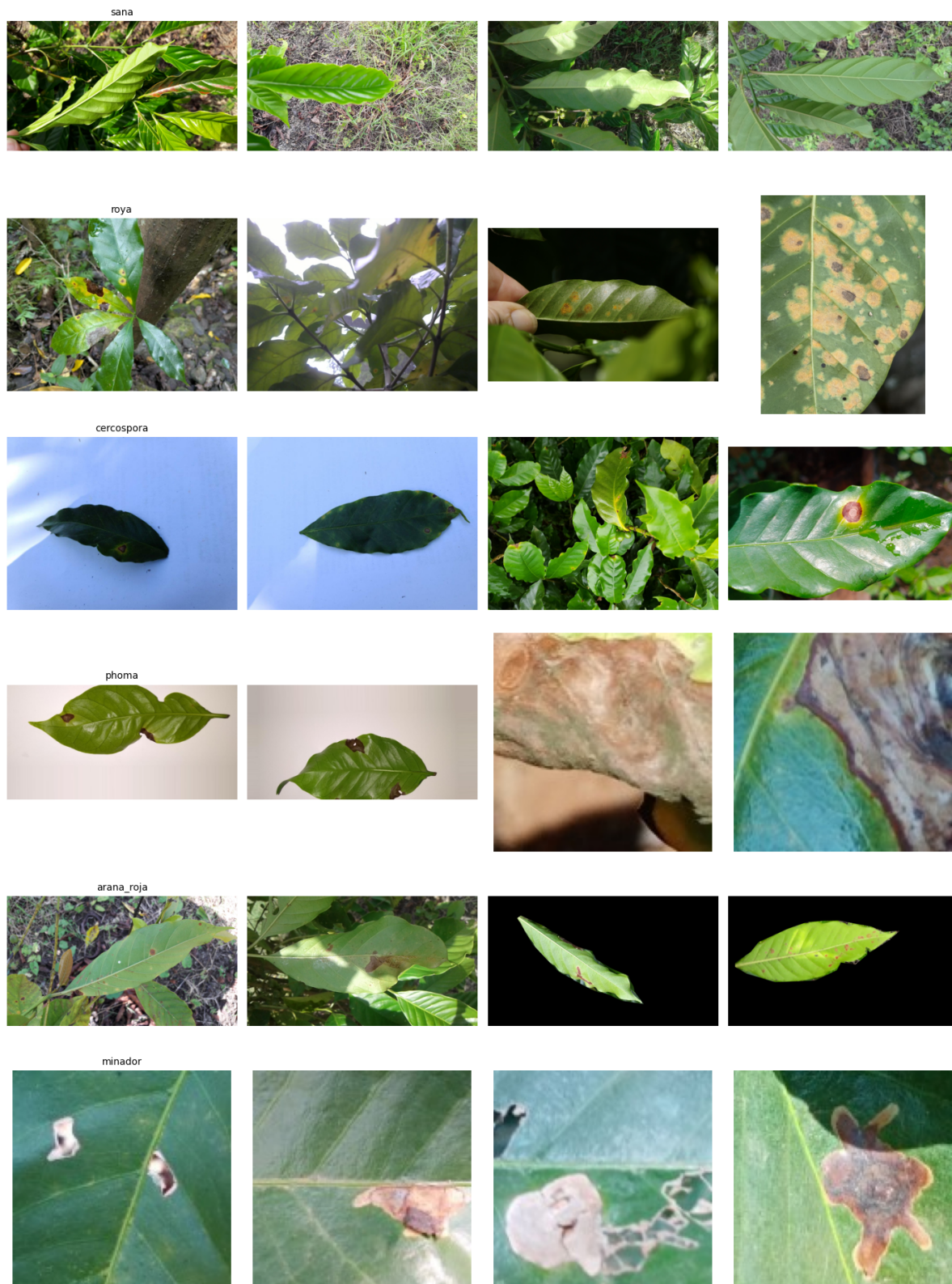


Figura 2: Ejemplos representativos de imágenes reales para cada una de las seis clases del dataset.

Antes de generar imágenes sintéticas se realizó una **revisión visual manual** de las imágenes reales, particularmente en las clases con menos datos, para descartar fotografías mal recoradas, con fondos atípicos o de calidad dudosa; generar variaciones sintéticas a partir de una imagen problemática solo multiplicaría ese mismo problema.

Se aplicó además un **balanceo del conjunto de entrenamiento** mediante generación de imágenes sintéticas (volteo horizontal, rotación, zoom/recorte, brillo, contraste y saturación), con una meta de referencia de 400 imágenes por clase (equivalente a 280 imágenes en el subconjunto de entrenamiento, dado el reparto 70%/15%/15%). Este balanceo se aplicó **únicamente sobre el conjunto de entrenamiento – nunca sobre validación o prueba** – para evitar fuga de información (*data leakage*) que inflara artificialmente las métricas. Al revisar el reparto real, la mayoría de las clases ya alcanzaba o superaba la meta de forma natural; la única clase que requirió imágenes sintéticas fue **araña roja** (248 imágenes reales + 32 sintéticas = 280 en entrenamiento), tal como se detalla en la Tabla 3.

Clase	Reales (train)	Sintéticas generadas	Total en train
Cercospora	411	0	411
Phoma	350	0	350
Minador	350	0	350
Roya	281	0	281
Sana	280	0	280
Araña roja	248	32	280
Total	1 920	32	1 952

Tabla 3: Balanceo del conjunto de entrenamiento mediante imágenes sintéticas. Solo la clase araña roja no alcanzaba la meta de 280 imágenes de forma natural.

Es importante señalar que **roya** ya contaba con cerca de 400 imágenes reales en el dataset original y, aun así, resultó ser una de las clases con menor desempeño (ver sección 9); su confusión con sana y araña roja en la matriz de confusión sugiere un problema de **solapamiento visual entre clases** más que de cantidad de datos, razón por la que la mejora de esta clase se abordó posteriormente mediante una ronda de ajuste fino enfocada (sección 9) y no mediante más datos sintéticos.

6.3 Algoritmos y técnicas de Inteligencia Artificial implementadas

- ❑ **Visión artificial (Computer Vision)** para el procesamiento y análisis de las imágenes de las hojas.
- ❑ **Aprendizaje profundo (Deep Learning)** mediante **Redes Neuronales Convolu-**

cionales (CNN).

- ❑ **Transfer Learning**, con comparación formal entre **MobileNetV2** y **EfficientNetB0**, entrenadas bajo el mismo protocolo de dos fases (ver sección 9).
- ❑ **Data augmentation** en tiempo de entrenamiento y generación de imágenes sintéticas *offline* para el balanceo de clases.
- ❑ **Grad-CAM** para interpretabilidad y diagnóstico de errores de clasificación.
- ❑ Definición de un **umbral de decisión** sobre la confianza de la predicción, para distinguir un diagnóstico confiable de uno no concluyente.

6.4 Herramientas y tecnologías empleadas

Categoría	Herramienta / Tecnología
Lenguaje de programación	Python
Framework de Deep Learning	TensorFlow / Keras
Procesamiento de imágenes	Pillow, OpenCV
Análisis numérico y de datos	NumPy, pandas
Visualización	Matplotlib, Seaborn
Métricas y evaluación	scikit-learn
Entrenamiento y experimentación	Google Colab (GPU gratuita)
Interfaz de usuario	Streamlit
Alojamiento del modelo entrenado	GitHub (repositorio del proyecto)
Recomendaciones mediante LLM	API de Groq (API key), consumida directamente desde Streamlit
Control de versiones	Git y GitHub
Documentación y análisis	Jupyter Notebook

Tabla 4: Herramientas y tecnologías utilizadas en el desarrollo del proyecto.

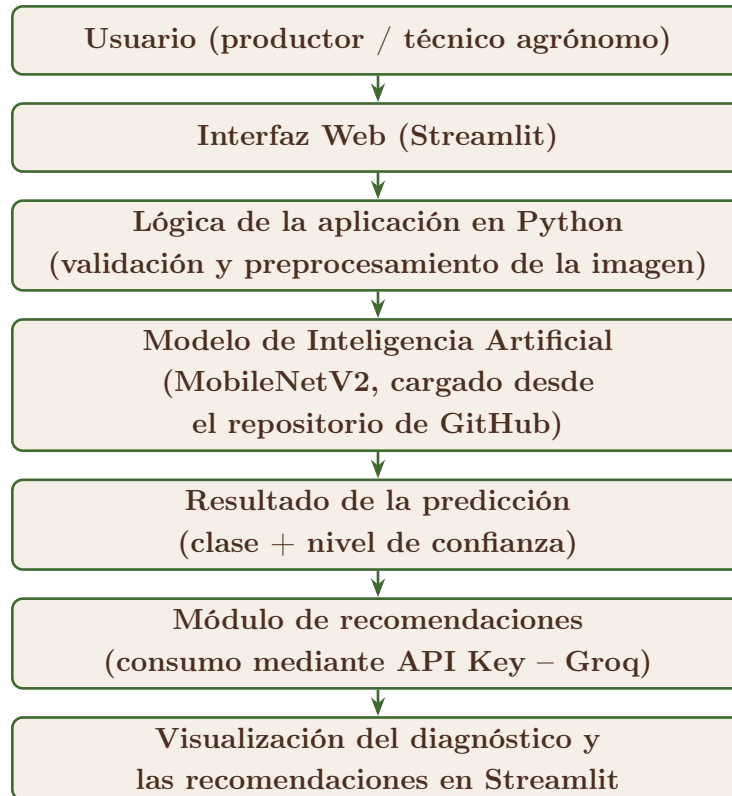
7 Diseño de la solución

7.1 Arquitectura del sistema

La aplicación se organiza en cuatro capas: interfaz de usuario, lógica de la aplicación, modelo de Inteligencia Artificial y módulo de recomendaciones. El modelo de visión (MobileNetV2, entrenado y evaluado según se describe en la sección 9) se **almacena versionado en el repositorio de GitHub del proyecto** y se carga directamente dentro de la aplicación de Streamlit al iniciar el servicio, sin necesidad de un servicio de inferencia independiente. El

módulo de recomendaciones, en cambio, se consume como un servicio externo mediante una **API key** (API de Groq), ya que corresponde a un modelo de lenguaje de terceros.

7.2 Diagrama de funcionamiento



7.3 Base de datos

El proyecto no requiere una base de datos relacional tradicional. Los activos de datos del sistema son: (1) el conjunto de imágenes organizado por clase, utilizado únicamente en la fase de entrenamiento; (2) el modelo entrenado, almacenado como archivo (`.keras` / `.h5`) en el repositorio de GitHub; y (3) un archivo de configuración (`config_app.json`) que define el orden de las clases, el tamaño de imagen esperado y el umbral de decisión, garantizando que la aplicación interprete correctamente la salida del modelo.

7.4 Flujo del sistema

1. El usuario sube o captura una fotografía de una hoja de café desde la interfaz de Streamlit.
2. La aplicación valida el formato de la imagen y la redimensiona/normaliza exacta-

mente igual que en el entrenamiento (224×224 píxeles, preprocesamiento propio de MobileNetV2).

3. El modelo, cargado en memoria desde el archivo del repositorio, genera una predicción (clase y probabilidad por cada una de las seis categorías).
4. Se contrasta la confianza de la predicción contra el umbral de decisión (0.60, ver sección 9): si es suficiente, se acepta el diagnóstico; si no, se informa al usuario que el resultado no es concluyente.
5. Si el diagnóstico fue aceptado, la aplicación construye un *prompt* con la clase detectada y realiza una solicitud a la API de Groq, autenticada mediante API key, para obtener recomendaciones de manejo agronómico.
6. El resultado final (diagnóstico + recomendaciones) se presenta al usuario en la misma interfaz.

8 Desarrollo e implementación

8.1 Módulos principales

El proyecto se organiza en tres módulos principales, documentados de forma reproducible en un *notebook* de Jupyter:

- ❑ **Módulo de datos:** descarga, extracción, organización, división (entrenamiento/validación/prueba) y balanceo del dataset.
- ❑ **Módulo de modelado y comparación de arquitecturas:** construcción, entrenamiento (dos fases: extracción de características y *fine-tuning*) y evaluación comparativa de las arquitecturas candidatas.
- ❑ **Módulo de diagnóstico e interpretabilidad:** matriz de confusión, curvas ROC, reporte de clasificación por clase y mapas de calor Grad-CAM, utilizados para orientar las mejoras del modelo.
- ❑ **Aplicación web (app.py):** interfaz de Streamlit, carga del modelo desde el repositorio, preprocesamiento de la imagen del usuario y consumo de la API de recomendaciones.

8.2 Fragmentos de código relevantes

El siguiente fragmento muestra la construcción del modelo mediante *transfer learning*, congelando la base preentrenada durante la primera fase de entrenamiento:

Listing 1: Construcción del modelo (transfer learning, fase 1)

```

1 base_model = MobileNetV2(input_shape=IMG_SIZE + (3,),
2                           include_top=False, weights="imagenet")
3 base_model.trainable = False # Fase 1: base congelada
4
5 inputs = keras.Input(shape=IMG_SIZE + (3,))
6 x = base_model(inputs, training=False)
7 x = layers.GlobalAveragePooling2D()(x)
8 x = layers.Dropout(0.3)(x)
9 x = layers.Dense(128, activation="relu")(x)
10 x = layers.Dropout(0.2)(x)
11 outputs = layers.Dense(NUM_CLASES, activation="softmax")(x)
12 model = keras.Model(inputs, outputs, name="AgroDetect_MobileNetV2")

```

El siguiente fragmento corresponde al cálculo del mapa de calor Grad-CAM, utilizado para verificar en qué región de la imagen se enfoca el modelo al predecir:

Listing 2: Cálculo del mapa de calor Grad-CAM

```

1 def obtener_mapa_gradcam(img_array_preprocesada, indice_clase=None):
2     with tf.GradientTape() as tape:
3         salida_conv = base_model(img_array_preprocesada, training=False)
4         tape.watch(salida_conv)
5         x = salida_conv
6         for layer in model.layers:
7             if layer is base_model or isinstance(layer, keras.layers.
8               InputLayer):
9                 continue
10            x = layer(x, training=False)
11        predicciones = x
12        if indice_clase is None:
13            indice_clase = tf.argmax(predicciones[0])
14            canal_clase = predicciones[:, indice_clase]
15
16        gradientes = tape.gradient(canal_clase, salida_conv)
17        pesos = tf.reduce_mean(gradientes, axis=(0, 1, 2))
18        mapa_calor = salida_conv[0] @ pesos[..., tf.newaxis]
19        mapa_calor = tf.maximum(tf.squeeze(mapa_calor), 0)
20        return (mapa_calor / (tf.reduce_max(mapa_calor) + 1e-8)).numpy()

```

8.3 Justificación de las decisiones técnicas

- ❑ **Comparación de arquitecturas en vez de elegir una a priori:** se entrenaron y evaluaron formalmente MobileNetV2 y EfficientNetB0 bajo el mismo protocolo de dos

fases, para tomar una decisión respaldada por evidencia y no solo por intuición.

- ❑ **Criterio de selección combinado (desempeño + tamaño):** dado que el modelo final se aloja en GitHub y se carga dentro de una aplicación con recursos limitados (Streamlit Community Cloud), la arquitectura ganadora se eligió ponderando el F1-score *y* el número de parámetros (70 %/30 %), no solo el F1-score máximo.
- ❑ **Balanceo del dataset solo en entrenamiento:** aplicar aumento de datos también sobre validación/prueba habría inflado artificialmente las métricas al filtrar información entre conjuntos.
- ❑ **Uso de Grad-CAM antes de seguir ajustando hiperparámetros:** permitió distinguir errores causados por brecha de dominio (fondo, iluminación, encuadre distintos a los del dataset) de errores causados por solapamiento visual genuino entre clases con síntomas similares.
- ❑ **Modelo servido desde GitHub en vez de una API de inferencia propia:** al ser un modelo liviano (MobileNetV2, apenas 2.42 millones de parámetros), cargarlo directamente dentro de la aplicación de Streamlit simplifica el despliegue y el mantenimiento, evitando la complejidad adicional de operar un servicio de inferencia independiente.

9 Resultados obtenidos

9.1 Comparación de arquitecturas

En una etapa previa del proyecto se había evaluado también **ResNet50**; esa arquitectura se descartó de la comparación final por ser demasiado pesada (cerca de 25 millones de parámetros) para un despliegue viable en Streamlit Community Cloud, aun cuando había mostrado buen F1-score en pruebas anteriores. Por ello, la comparación formal documentada en este notebook se centró en dos arquitecturas livianas y modernas: **MobileNetV2** y **EfficientNetB0**, entrenadas bajo el mismo protocolo (fase 1: cabeza de clasificación con base congelada; fase 2: *fine-tuning* con un bloque de capas superiores descongelado). La Tabla 5 y la Figura 3 resumen los resultados sobre el conjunto de prueba.

Arquitectura	Exactitud	Precisión	Sensibilidad	F1-score	Parámetros	Tiempo
MobileNetV2	0.862	0.882	0.845	0.863	2.42 M	34.1 min
EfficientNetB0	0.836	0.902	0.775	0.834	4.21 M	47.8 min

Tabla 5: Comparación de arquitecturas sobre el conjunto de prueba (414 imágenes).

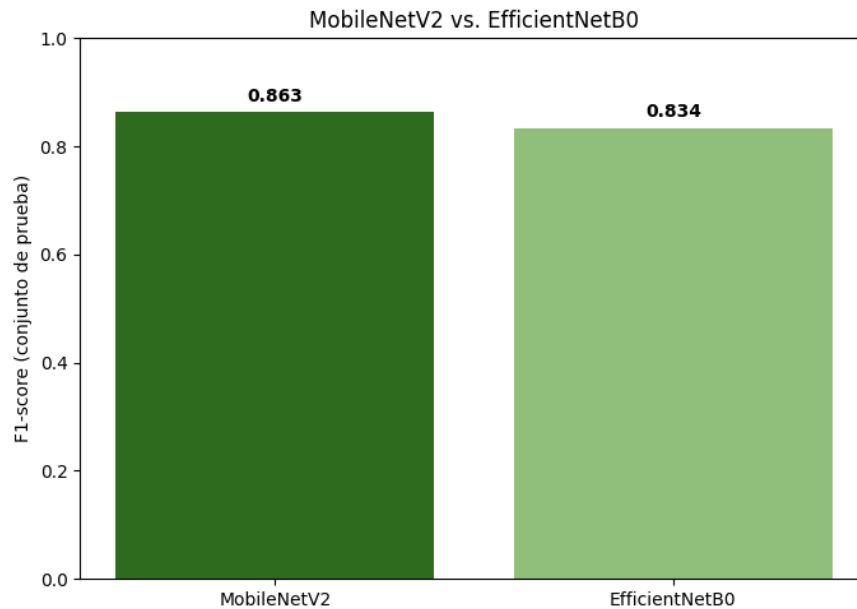


Figura 3: Comparación de F1-score entre MobileNetV2 y EfficientNetB0 sobre el conjunto de prueba.

MobileNetV2 obtuvo un F1-score ligeramente superior (0.863 frente a 0.834) y, además, casi la mitad de parámetros que EfficientNetB0 (2.42 frente a 4.21 millones), por lo que resultó la opción más conveniente tanto en desempeño como en viabilidad de despliegue. Al combinar ambos criterios mediante un puntaje ponderado (70 % F1-score, 30 % tamaño relativo del modelo), MobileNetV2 obtuvo la puntuación máxima (1.000 frente a 0.849 de EfficientNetB0) y fue seleccionada como arquitectura definitiva para el resto del proyecto.

9.2 Curvas de entrenamiento

La Figura 4 muestra la evolución de la exactitud y la pérdida durante las dos fases de entrenamiento de MobileNetV2 (cabeza congelada y *fine-tuning*). Las curvas de validación se mantienen cercanas a las de entrenamiento durante ambas fases, sin señales marcadas de sobreajuste, lo que respalda el uso de *data augmentation* y *early stopping* como mecanismos de regularización.

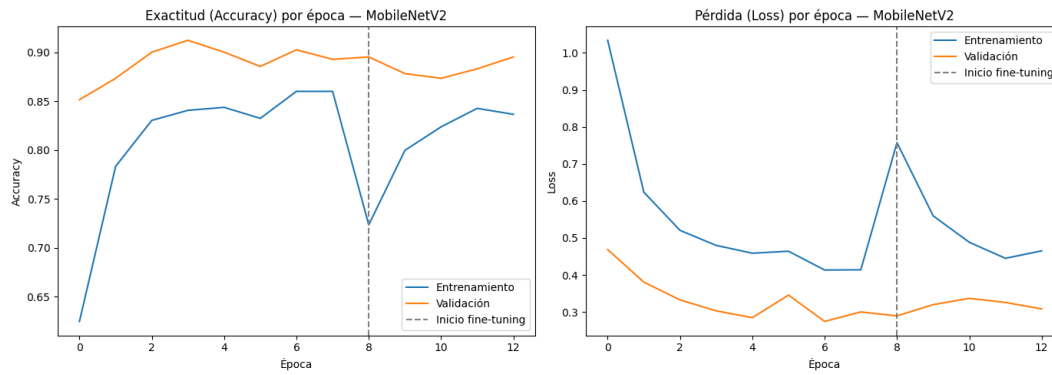


Figura 4: Curvas de exactitud y pérdida durante el entrenamiento de MobileNetV2 (fases 1 y 2 combinadas).

9.3 Desempeño del modelo final

Tras la evaluación inicial del modelo ganador (exactitud de 0.862 y F1-score macro de 0.850 sobre el conjunto de prueba), se aplicó una ronda adicional de ajuste fino enfocada (detallada en la sección 9.4), que llevó el desempeño final a una **exactitud de 0.874** y un **F1-score macro de 0.863** sobre el conjunto de prueba (414 imágenes). La Tabla 6 y la Figura 5 presentan el detalle por clase y la matriz de confusión del modelo final.

Clase	Precisión	Sensibilidad	F1-score	Soporte
Sana	0.843	0.983	0.908	60
Roya	0.814	0.787	0.800	61
Cercospora	0.825	0.955	0.885	89
Phoma	0.944	0.893	0.918	75
Araña roja	0.850	0.630	0.723	54
Minador	0.972	0.920	0.945	75
Promedio macro	0.875	0.861	0.863	414

Tabla 6: Reporte de clasificación por clase del modelo final (MobileNetV2, tras la ronda de ajuste fino).

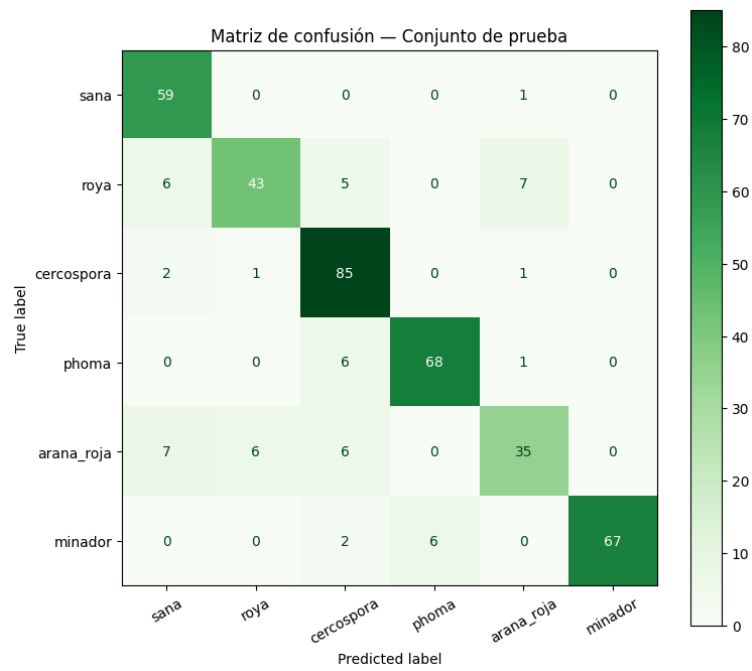


Figura 5: Matriz de confusión del modelo final sobre el conjunto de prueba.

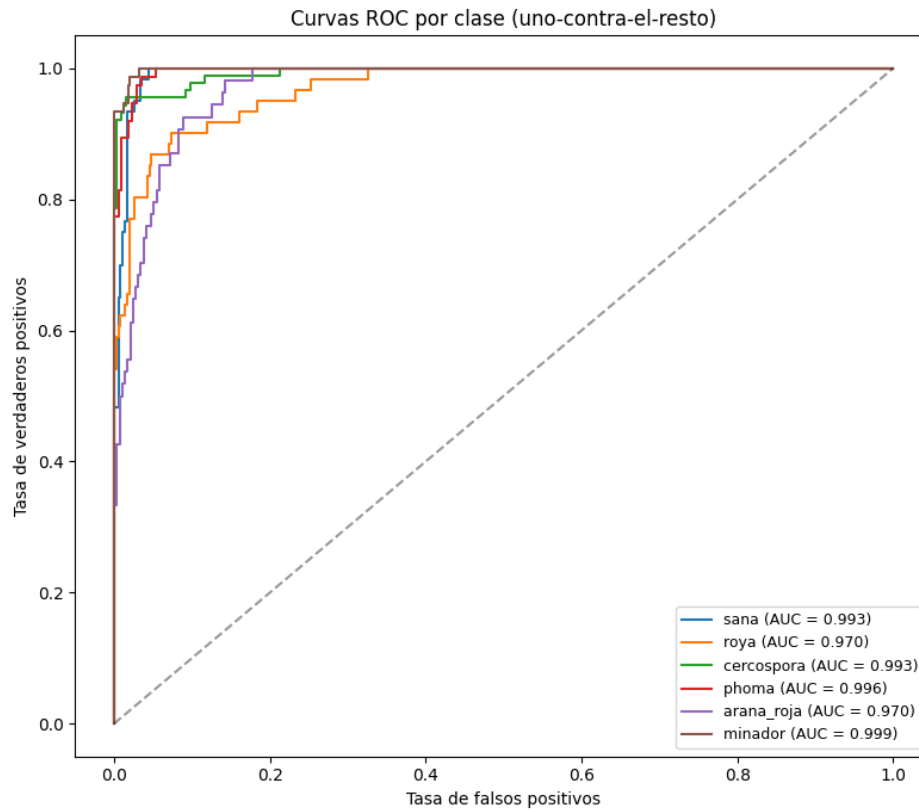


Figura 6: Curvas ROC y AUC por clase del modelo final sobre el conjunto de prueba.

La clase con mejor desempeño es **minador** ($F1 = 0.945$), mientras que **araña roja** sigue siendo la más débil ($F1 = 0.723$), principalmente por baja sensibilidad: el modelo deja de reconocer un número considerable de casos reales de araña roja, confundiéndolos con otras clases.

9.4 Interpretabilidad del modelo y ronda de ajuste fino (Grad-CAM)

Para verificar que el modelo aprendiera patrones visuales relevantes – y no atajos del dataset, como el fondo de la fotografía (*domain gap*) – se aplicó Grad-CAM sobre las clases con menor desempeño, en particular araña roja y cercospora. El análisis mostró de forma consistente que el modelo centra su atención en la hoja y sus lesiones la mayoría de las veces, descartando aprendizaje de atajos espurios. Se identificó además un patrón sistemático relevante para el diagnóstico: las imágenes correctamente clasificadas presentan daño distribuido en una porción amplia de la hoja, mientras que las clasificadas incorrectamente muestran daño concentrado en una mancha puntual, más difícil de diferenciar entre enfermedades de síntoma similar.

Grad-CAM sobre imágenes de araña_roja (rojo/amarillo = zona que más influyó en la predicción)

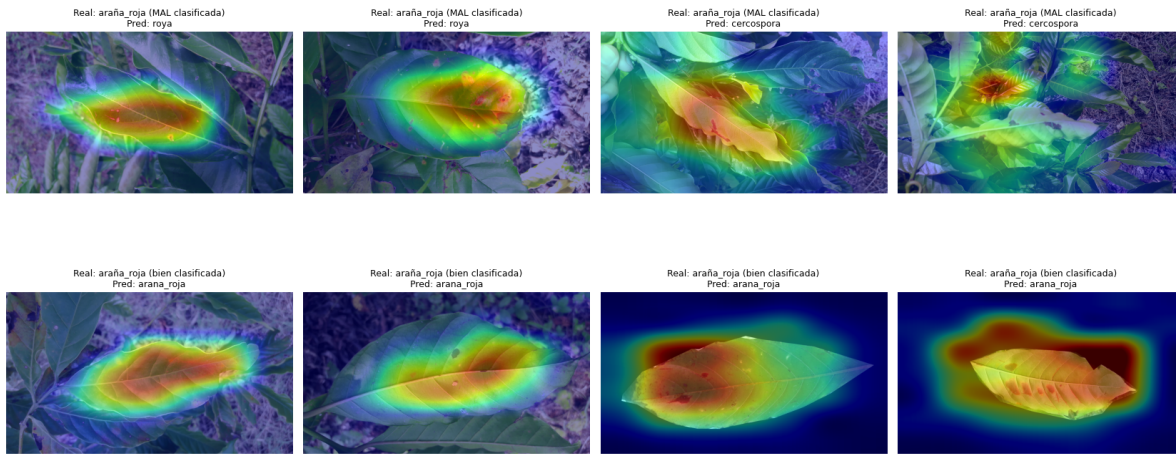


Figura 7: Mapas de calor Grad-CAM sobre imágenes de araña roja, mal y bien clasificadas.

Grad-CAM sobre imágenes de cercospora

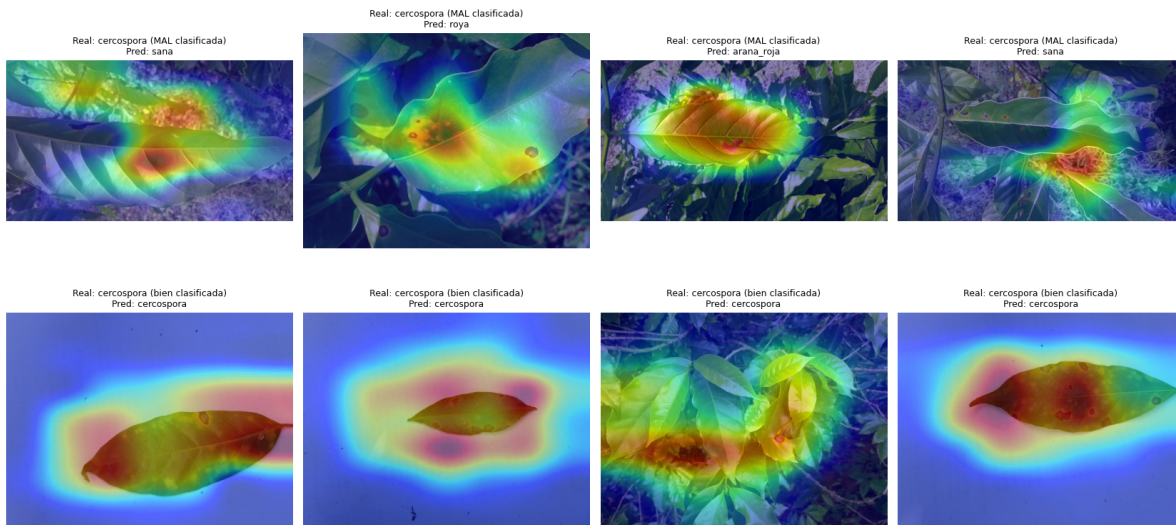


Figura 8: Mapas de calor Grad-CAM sobre imágenes de cercospora, mal y bien clasificadas.

A partir de este diagnóstico, en vez de ampliar el dataset se optó por una **ronda adicional de ajuste fino corta**, continuando desde los pesos ya entrenados del modelo ganador, con dos ajustes concretos: (1) se incrementó en $1.6\times$ el peso de la clase **roya** dentro de la función de pérdida, para penalizar más sus errores; y (2) se descongelaron 20 capas adicionales de la base convolucional (60 de 154 capas entrenables en total), permitiendo un ajuste fino extra

sobre los filtros de textura, con una tasa de aprendizaje muy baja (5×10^{-6}) durante 5 épocas adicionales. La Tabla 7 compara el desempeño por clase antes y después de esta ronda.

Clase	F1 antes	F1 después	Δ F1	Recall antes	Recall después
Sana	0.881	0.908	+0.027	0.983	0.983
Roya	0.775	0.800	+0.025	0.705	0.787
Cercospora	0.881	0.885	+0.005	0.955	0.955
Phoma	0.913	0.918	+0.005	0.907	0.893
Araña roja	0.707	0.723	+0.016	0.648	0.630
Minador	0.944	0.945	+0.002	0.893	0.920
F1 macro	0.850	0.863	+0.013	–	–

Tabla 7: Comparación por clase antes y después de la ronda de ajuste fino enfocada.

La mejora más marcada se observa en **roya** (F1 de 0.775 a 0.800, con un aumento de sensibilidad de casi 8 puntos porcentuales), que era precisamente la clase reforzada en la función de pérdida; el resto de clases mostró mejoras menores, atribuibles al ajuste fino adicional de las capas convolucionales compartidas. Al mejorar (o mantener) el F1 macro global, este modelo reforzado fue el adoptado como versión final para el resto del proyecto.

Nota de trazabilidad: en el *notebook* de entrenamiento, esta ronda de ajuste fino se documenta bajo el título “refinamiento enfocado en cercospora y araña roja”, pero el parámetro que efectivamente se configuró en el código (`CLASES_A_REFORZAR`) refuerza únicamente la clase **roya**. Los resultados y las cifras de esta sección reflejan lo que el código realmente ejecutó. Se recomienda revisar esta discrepancia entre la intención documentada y la configuración usada antes de reportar el proyecto como definitivo (ver sección 15).

9.5 Umbral de decisión

AgroDetect funciona como una primera línea de apoyo al diagnóstico, no como reemplazo del criterio del técnico agrónomo. Por ello se analizó la distribución de la confianza (probabilidad softmax de la clase predicha) del modelo sobre el conjunto de prueba, para definir un **umbral de decisión**: por debajo de este valor, la aplicación reporta un diagnóstico “incierto” en vez de forzar una clase.

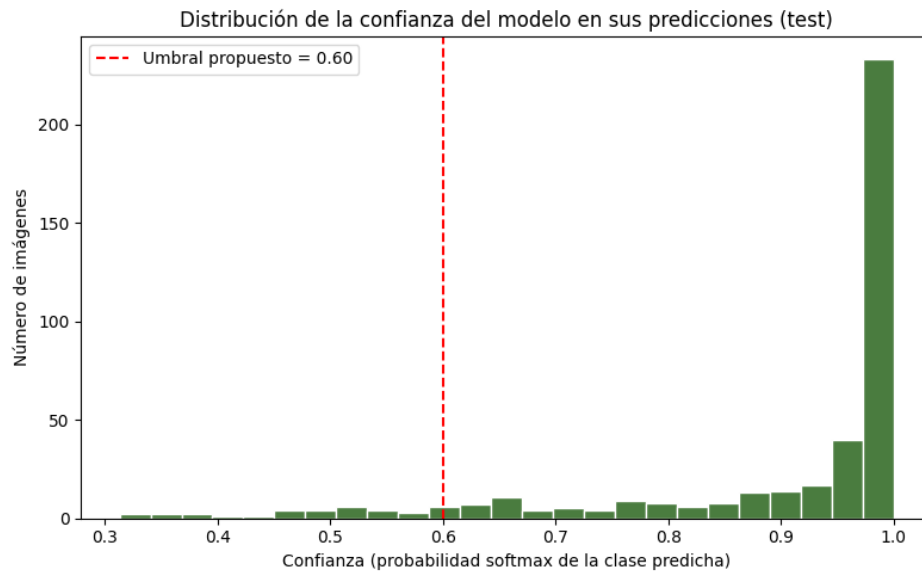


Figura 9: Distribución de la confianza del modelo en sus predicciones sobre el conjunto de prueba.

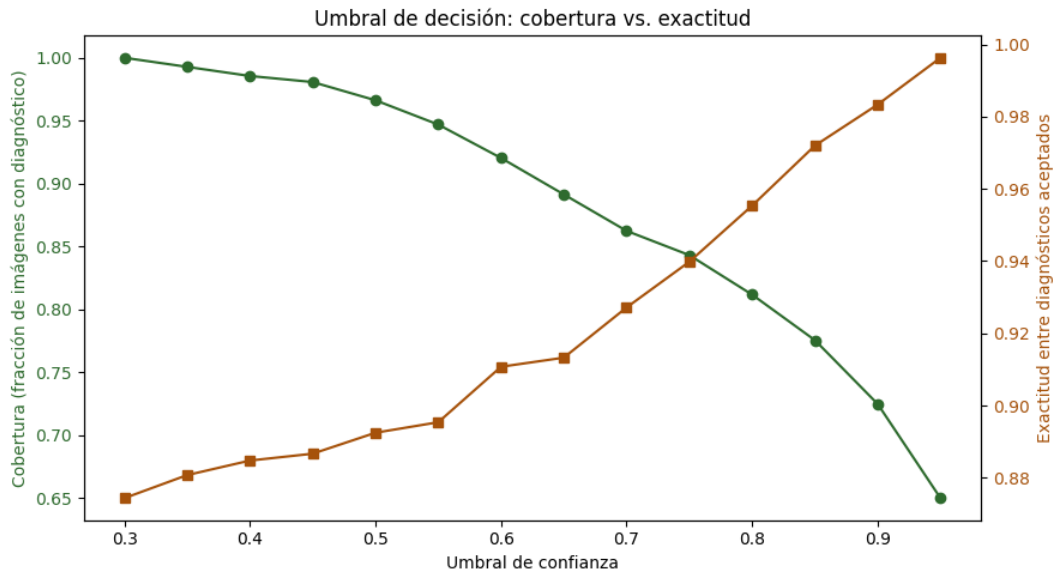


Figura 10: Cobertura (% de imágenes con diagnóstico) frente a exactitud entre los diagnósticos aceptados, para distintos umbrales de confianza.

Se seleccionó un umbral de **0.60**, que ofrece un equilibrio razonable entre cobertura y confiabilidad: con este valor, el sistema entrega un diagnóstico concluyente en el **92.0 %** de las imágenes de prueba (381 de 414), con una exactitud del **91.1 %** entre esos diagnósticos acep-

tados; el 8.0 % restante (33 imágenes) se reporta como diagnóstico incierto, recomendando consultar a un técnico del IHCAFE.

9.6 Pruebas realizadas

A lo largo del desarrollo se realizaron múltiples rondas de prueba y ajuste, entre ellas:

- ❑ Comparación formal de dos arquitecturas (MobileNetV2, EfficientNetB0), tras haber descartado previamente ResNet50 por su tamaño (25 millones de parámetros).
- ❑ Balanceo del conjunto de entrenamiento mediante imágenes sintéticas, aplicado únicamente a la clase araña roja (la única que no alcanzaba la meta de forma natural), precedido de una auditoría visual manual del dataset.
- ❑ Análisis de interpretabilidad Grad-CAM sobre las clases más débiles (araña roja y cercospora), seguido de una ronda de ajuste fino enfocada, adoptada solo tras confirmar mejora en el F1 macro (de 0.850 a 0.863).
- ❑ Calibración del umbral de decisión mediante el análisis de cobertura vs. exactitud, seleccionando 0.60 como valor de referencia para la aplicación.
- ❑ Pruebas del modelo con una fotografía real fuera del dataset, para verificar su capacidad de generalización más allá del conjunto de prueba: en un caso probado, el modelo asignó una confianza máxima de apenas 46 % (a la clase araña roja, seguida de roya con 35 %), por debajo del umbral de 0.60; el sistema reportó correctamente el diagnóstico como incierto en lugar de forzar una respuesta poco confiable, validando en la práctica el mecanismo de umbral de decisión.

10 Beneficios del proyecto

- ❑ **Diagnóstico más rápido y accesible**, sin necesidad de esperar la visita de un técnico especializado.
- ❑ **Identificación oportuna** de enfermedades y plagas, permitiendo decisiones de manejo fitosanitario más adecuadas y reduciendo pérdidas de cosecha.
- ❑ **Reducción de costos** asociados al uso indiscriminado de fungicidas y pesticidas, al facilitar un manejo más focalizado.
- ❑ **Democratización del conocimiento agronómico**, poniendo a disposición de cualquier productor con un teléfono inteligente una herramienta de diagnóstico preliminar.
- ❑ **Generación de datos útiles** para futuras investigaciones sobre la incidencia de enfer-

medades y plagas del café en la región.

11 Usuarios beneficiados

- ❑ **Pequeños y medianos productores de café** del departamento de Comayagua, principalmente aquellos con acceso limitado a asistencia técnica especializada.
- ❑ **Técnicos agrónomos y extensionistas** del IHCAFE y de cooperativas cafetaleras locales.
- ❑ **Cooperativas y asociaciones de caficultores** de la región.
- ❑ **Estudiantes y docentes** de carreras agronómicas o afines.

12 Escalabilidad de la solución

La separación entre la interfaz de Streamlit, el modelo de visión y el módulo de recomendaciones facilita la escalabilidad del sistema. El modelo puede reentrenarse o incluso reemplazarse por una arquitectura distinta actualizando únicamente el archivo alojado en GitHub, sin modificar el código de la aplicación. Si el número de usuarios creciera de forma que la carga del modelo dentro del propio proceso de Streamlit se vuelva un cuello de botella, la arquitectura permite migrar el modelo a un servicio de inferencia independiente (por ejemplo, expuesto mediante una API key propia) sin rediseñar la interfaz, ya que la lógica de la aplicación ya está desacoplada del resto del sistema.

13 Mantenimiento de la aplicación

13.1 Operación continua

Se establecerá un plan básico de monitoreo del funcionamiento de la aplicación (disponibilidad del servicio, tiempos de respuesta y errores de predicción reportados por los usuarios), utilizando los registros (*logs*) proporcionados por la plataforma de despliegue.

13.2 Estrategia de actualización del modelo

- ❑ Recolección periódica de nuevas imágenes aportadas por los propios usuarios (con su consentimiento), para ampliar y diversificar el dataset, priorizando las clases con menor desempeño (araña roja y roya).
- ❑ Reentrenamiento periódico del modelo incorporando los nuevos datos, siguiendo el mismo

protocolo de comparación de arquitecturas documentado en este informe.

- ❑ Versionado de los modelos entrenados en el repositorio de GitHub, junto con su archivo `config_app.json` y sus métricas, de manera que sea posible revertir a una versión anterior si una actualización reduce el desempeño.
- ❑ Revisión y actualización periódica del *prompt* y las reglas de negocio del módulo de recomendaciones, de forma independiente al modelo de visión.

13.3 Mantenimiento preventivo y correctivo

- ❑ **Preventivo:** revisiones periódicas del código, actualización de dependencias y pruebas de regresión antes de cada despliegue de una nueva versión del modelo.
- ❑ **Correctivo:** atención a errores reportados por los usuarios mediante un canal de retroalimentación simple integrado en la propia aplicación.

13.4 Respaldo de la información

Respaldo periódico del código fuente, el *notebook* de entrenamiento y los modelos versionados en el repositorio de GitHub, y del dataset e imágenes recolectadas.

14 Conclusiones

- ❑ Se desarrolló y evaluó un sistema de clasificación de enfermedades y plagas foliares del café, alcanzando un F1-score macro de 0.863 y una exactitud de 0.874 sobre el conjunto de prueba, mediante *transfer learning* con la arquitectura MobileNetV2.
- ❑ La comparación formal de arquitecturas (MobileNetV2 y EfficientNetB0, tras descartar ResNet50 por su tamaño en una etapa previa) permitió tomar una decisión informada, priorizando no solo el desempeño sino también la viabilidad de despliegue del modelo dentro de la aplicación.
- ❑ El análisis de interpretabilidad mediante Grad-CAM resultó clave para descartar problemas de brecha de dominio y para diagnosticar la causa real de la confusión entre clases (daño puntual vs. difuso), orientando una ronda de ajuste fino basada en evidencia en lugar de prueba y error.
- ❑ La ronda de ajuste fino enfocada mejoró el F1 macro del modelo de 0.850 a 0.863, con la mejora más notable en la clase roya, que fue la efectivamente reforzada en la función de pérdida.

- ❑ La calibración de un umbral de decisión (0.60) permite que la aplicación distinga diagnósticos confiables de inciertos, entregando un diagnóstico concluyente en el 92 % de los casos de prueba con una exactitud del 91 % entre ellos, y evitando presentar como certeras predicciones de baja confianza – comportamiento que se validó también con una fotografía externa al dataset.
- ❑ El sistema final integra un modelo de visión alojado y versionado en GitHub, cargado directamente por la aplicación de Streamlit, junto con un módulo de recomendaciones consumido mediante API key, cumpliendo con el objetivo de ofrecer una herramienta accesible, liviana y de bajo costo de despliegue.

15 Recomendaciones

- ❑ Continuar ampliando el dataset de **araña roja**, la clase con menor desempeño ($F1 = 0.723$), priorizando imágenes en primer plano y de distintas etapas de la infestación, ya que es la única clase que actualmente requiere balanceo sintético.
- ❑ Revisar y corregir la discrepancia detectada entre la intención documentada de la ronda de ajuste fino (reforzar cercospora y araña roja) y la clase efectivamente reforzada en el código (`CLASES_A_REFORZAR = ["roya"]`), y documentar explícitamente qué clase(s) se refuerzan en cada ronda futura para evitar ambigüedades en el reporte de resultados.
- ❑ Evaluar la incorporación de fotografías tomadas directamente en campo por los propios usuarios (con su consentimiento) para reducir la brecha entre las condiciones del dataset y las condiciones reales de uso.
- ❑ Repetir periódicamente el análisis Grad-CAM tras cada reentrenamiento, como parte del proceso de control de calidad del modelo, y no solo como diagnóstico puntual.
- ❑ Re-calibrar el umbral de decisión cada vez que se reentrene el modelo, dado que la distribución de confianza puede cambiar con nuevos datos.
- ❑ Documentar y versionar cada modelo entrenado en GitHub junto con sus métricas, para poder comparar objetivamente si una actualización futura realmente mejora el sistema.
- ❑ Evaluar en una fase posterior la migración del modelo a un servicio de inferencia propio si el volumen de usuarios lo justifica, aprovechando que la arquitectura actual ya desacopla la interfaz del modelo.

16 Referencias bibliográficas

-
- barakobama. (s.f.). *Barakobama Coffee Leaf Disease* [Conjunto de datos]. Roboflow Universe. <https://universe.roboflow.com/barakobama/barakobama-coffee-leaf-disease>
- biniyamyoseph. (s.f.). *Ethiopian Coffee Leaf Disease* [Conjunto de datos]. Kaggle. <https://www.kaggle.com/datasets/biniyamyoseph/ethiopian-coffee-leaf-disease>
- coffee-leaf-detection. (s.f.). *ROCC* [Conjunto de datos]. Roboflow Universe. <https://universe.roboflow.com/coffee-leaf-detection/rocc>
- dame-23kub. (s.f.). *Coffee Leaf Disease* [Conjunto de datos]. Roboflow Universe. <https://universe.roboflow.com/dame-23kub/coffee-leaf-disease-kc1ca>
- izzaiqbal. (s.f.). *Capstone Project* [Conjunto de datos]. Kaggle. <https://www.kaggle.com/datasets/izzaiqbal/capstone-project>
- lance-eugene. (s.f.). *Red Spider Mite* [Conjunto de datos]. Roboflow Universe. <https://universe.roboflow.com/lance-eugene/red-spider-mite>
- letang-dataset. (s.f.). *Coffee Dataset* [Conjunto de datos]. Roboflow Universe. <https://universe.roboflow.com/letang-dataset/coffee-dataset-gh98d>
- noamaanabdulazeem. (s.f.). *JMuBEN Coffee Dataset* [Conjunto de datos]. Kaggle. <https://www.kaggle.com/datasets/noamaanabdulazeem/jmuben-coffee-dataset>
- Parraga-Alava, J., Cusme, K., Loor, A., & Santander, E. (2019). *RoCoLe: A robusta coffee leaf images dataset* [Conjunto de datos] (Versión 2). Mendeley Data. <https://data.mendeley.com/datasets/c5yvn32dzg/2>
- Sankalana, N. (s.f.). *RoCoLe – A Robusta Coffee Leaf Images Dataset* [Conjunto de datos]. Kaggle. <https://www.kaggle.com/datasets/nimalsankalana/rocole-a-robusta-coffee-leaf-images-dataset>
- Velásquez, D., Sánchez, A., Sarmiento, S., Velásquez, C., Toro, M., Montoya, E., Maiza, M., Sierra, B., & Trefftz, H. (2020). *Coffee Leaf Rust dataset* [Conjunto de datos]. GitHub. <https://github.com/dvelaren/clr-dataset/tree/master/data/0>
- yolonasv1. (s.f.). *Coffee Leaf Disease* [Conjunto de datos]. Roboflow Universe. <https://universe.roboflow.com/yolonasv1/coffee-leaf-disease-o9dbv>
-

- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). *Grad-CAM: Visual explanations from deep networks via gradient-based localization*. En *Proceedings of the IEEE International Conference on Computer Vision* (pp. 618-626).
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). *MobileNetV2: Inverted residuals and linear bottlenecks*. En *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4510-4520).

17 Anexos

- ❑ **Repositorio del proyecto** (código fuente, *notebook* de entrenamiento, modelo versionado y aplicación de Streamlit): <https://github.com/Izamar-0302/AgroDetect>
- ❑ **Archivo de configuración de la aplicación** (`config_app.json`): incluye el orden de clases, tamaño de imagen esperado, umbral de decisión y métricas de la comparación de arquitecturas, generado automáticamente al final del *notebook* de entrenamiento.
- ❑ **Informe en Overleaf** (código latex): https://www.overleaf.com/9164767963fkrbm_tvpchvq#b87b3a
- ❑ **Url de streamlit** (Despliegue de app): <https://agrodetect-grup4.streamlit.app/>